Prática — Gerador Blum Blum Shub e suíte de testes NIST

DCC075 – Segurança em Sistemas Computacionais · UFJF · Lucas Gonçalves Rocha · Matrícula 202265187C · Individual · Outubro de 2026

1. Valores utilizados

p	**102407**
q	**98999**
s (semente)	**5333145159** — sorteada com `SecureRandom`
n = p · q	10138190593 (≈ 2^{33})
Bits gerados e testados	1.000.000
Período da sequência	1.267.198.298 passos (muito maior que os bits testados)

2. O algoritmo e as premissas

O Blum Blum Shub é um gerador pseudoaleatório criptográfico: além de parecer aleatório, o próximo bit não deve ser previsível sem fatorar n. Ele eleva um número ao quadrado módulo n repetidamente e usa a paridade de cada resultado:

```
n = p · q                 com p e q primos e p ≡ q ≡ 3 (mod 4)
x0 = s² mod n             com 1 < s < n e mdc(s, n) = 1
x(i+1) = x(i)² mod n
bit(i) = x(i) mod 2       (bit menos significativo)
```

Premissa	Por quê	Verificação
p e q primos, p ≠ q	a segurança vem da dificuldade de fatorar n; com p = q, basta extrair a raiz	✓
p ≡ q ≡ 3 (mod 4)	torna x → x² uma permutação dos resíduos quadráticos (“primos de Blum”); a sequência é puramente periódica	102407 mod 4 = 3; 98999 mod 4 = 3 ✓
1 < s < n e mdc(s, n) = 1	com fator comum, a sequência degenera	mdc = 1 ✓
p, q > 10000	exigência do enunciado	✓

Além das premissas, p e q são primos “especiais” (102407 = 2·51203 + 1 com 51203 = 2·25601 + 1; 98999 = 2·49499 + 1 com 49499 = 2·24749 + 1, todos primos), condição do artigo original para maximizar o período, que divide λ(λ(n)). O código confirma que a sequência não volta ao x_0 dentro do 1.000.000 de bits gerados. Exemplo pequeno, feito à mão e usado nos testes automatizados: com p = 7, q = 11 e s = 3, x_0 = 9 → 4 → 16 → 25 → 9 (período 4), e os bits são 0, 0, 1, 1, repetindo.

```
lucas@MacBook-Air-de-Lucas dcc075 % jshell
|  Welcome to JShell -- Version 21.0.12.1
|  For an introduction type: /help intro

jshell> import java.math.BigInteger;
   ...> var p = new BigInteger("102407");
   ...> var q = new BigInteger("98999");
   ...> var s = new BigInteger("5333145159");
   ...> var n = p.multiply(q);
   ...> var quatro = BigInteger.valueOf(4);
   ...> System.out.println("p primo? " + p.isProbablePrime(50) + " | p mod 4 = " + p.mod(quatro));
   ...> System.out.println("q primo? " + q.isProbablePrime(50) + " | q mod 4 = " + q.mod(quatro));
   ...> System.out.println("p > 10000? " + (p.intValue() > 10000) + " | q > 10000? " + (q.intValue() > 10000) + " | p !=
 q? " + !p.equals(q));
   ...> System.out.println("n = " + n);
   ...> System.out.println("1 < s < n? " + (s.compareTo(BigInteger.ONE) > 0 && s.compareTo(n) < 0) + " | mdc(s, n) = " +
 s.gcd(n));
p ==> 102407
q ==> 98999
s ==> 5333145159
n ==> 10138190593
quatro ==> 4
p primo? true | p mod 4 = 3
q primo? true | q mod 4 = 3
p > 10000? true | q > 10000? true | p != q? true
n = 10138190593
1 < s < n? true | mdc(s, n) = 1
```

Figura 1 — Verificação das premissas de p, q e s no jshell (Java 21).



Figura 2 — Tela da aplicação (/bbs): todas as premissas atendidas, 499.914 zeros e 500.086 uns, sem repetição no período.

3. Código do algoritmo (BlumBlumShub.java)

Implementado em Java 21, dentro da aplicação Spring Boot da disciplina. O `BigInteger` é necessário porque, com $n \approx 10^{10^6}$, x^2 chega a $\approx 10^{2 \cdot 10^6}$ e estouraria o `long` em silêncio (CWE-190). O código completo, com a tela e os testes, está no repositório: github.com/lucasdu01/UFJF-DCC075-Seguranca (pacote `bbs`).

```
package br.ufjf.dcc075.bbs;

import org.springframework.stereotype.Service;

import java.math.BigInteger;
import java.util.List;
import java.util.OptionalLong;

import static java.math.BigInteger.ONE;

/**
```

```java
 * Gerador pseudoaleatório Blum Blum Shub (BBS).
 * <p>
 * n = p * q, com p e q primos e p ≡ q ≡ 3 (mod 4)  ("primos de Blum")
 * x0 = s² mod n, com 1 < s < n e mdc(s, n) = 1
 * x(i+1) = x(i)² mod n
 * bit(i) = x(i) mod 2  (bit menos significativo)
 * <p>
 * A segurança vem da dificuldade de fatorar n: prever o próximo bit é tão
 * difícil quanto fatorar n. Com p e q de 5 dígitos (como nesta atividade),
 * n é fatorado em milissegundos, então este gerador é didático, não seguro.
 */
@Service
public class BlumBlumShub {

    private static final BigInteger TRES = BigInteger.valueOf(3);
    private static final BigInteger QUATRO = BigInteger.valueOf(4);

    /** Certeza do teste de Miller-Rabin: chance de erro < 2^-50. */
    private static final int CERTEZA_PRIMO = 50;

    /** Uma premissa do algoritmo e se ela foi atendida (exibida na tela com ✓/✗). */
    public record Premissa(String descricao, boolean atendida) {}

    /** Confere todas as premissas matemáticas do BBS para os valores informados. */
    public List<Premissa> verificar(BigInteger p, BigInteger q, BigInteger s) {
        BigInteger n = p.multiply(q);
        return List.of(
                new Premissa("p é primo", p.isProbablePrime(CERTEZA_PRIMO)),
                new Premissa("q é primo", q.isProbablePrime(CERTEZA_PRIMO)),
                new Premissa("p ≠ q (se p = q, n = p² e basta tirar a raiz para fatorar)", !p.equals(q)),
                new Premissa("p ≡ 3 (mod 4)", p.mod(QUATRO).equals(TRES)),
                new Premissa("q ≡ 3 (mod 4)", q.mod(QUATRO).equals(TRES)),
                new Premissa("1 < s < n", s.compareTo(ONE) > 0 && s.compareTo(n) < 0),
                new Premissa("mdc(s, n) = 1", s.gcd(n).equals(ONE))
        );
    }

    /**
     * Gera {@code quantidade} bits como uma String de '0' e '1'.
     *
     * @throws IllegalArgumentException se alguma premissa não for atendida
     */
    public String gerarBits(BigInteger p, BigInteger q, BigInteger s, int quantidade) {
        exigirPremissas(p, q, s);
        if (quantidade < 1) {
            throw new IllegalArgumentException("A quantidade de bits deve ser positiva.");
        }

        BigInteger n = p.multiply(q);
        BigInteger x = s.multiply(s).mod(n);          // x0 = s² mod n

        StringBuilder bits = new StringBuilder(quantidade);
        for (int i = 0; i < quantidade; i++) {
            x = x.multiply(x).mod(n);                 // x(i+1) = x(i)² mod n
            bits.append(x.testBit(0) ? '1' : '0');    // bit menos significativo
        }
        return bits.toString();
    }

    /**
     * Conta quantos passos a sequência leva para voltar ao x0 (o período).
     * <p>
     * Com primos de Blum, x -> x² é uma permutação dos resíduos quadráticos, então
     * a sequência é puramente periódica: sempre volta exatamente ao x0, sem "cauda"
     * antes do ciclo. Basta contar os passos até x == x0.
     * <p>
     * O período real pode ter bilhões de passos; para não iterar tudo, a contagem
     * para em {@code limite}. Isso basta para garantir que N bits gerados não se repetem.
     *
     * @return o período, ou vazio se ele for maior que {@code limite}
     */
    public OptionalLong periodo(BigInteger p, BigInteger q, BigInteger s, long limite) {
        exigirPremissas(p, q, s);

        BigInteger n = p.multiply(q);
        BigInteger x0 = s.multiply(s).mod(n);
        BigInteger x = x0;
        for (long i = 1; i <= limite; i++) {
            x = x.multiply(x).mod(n);
```

```
            if (x.equals(x0)) {
                return OptionalLong.of(i);
            }
        }
        return OptionalLong.empty();
    }

    /** Lança exceção com a primeira premissa que falhar, em vez de gerar uma sequência inválida. */
    private void exigirPremissas(BigInteger p, BigInteger q, BigInteger s) {
        verificar(p, q, s).stream()
                .filter(premissa -> !premissa.atendida())
                .findFirst()
                .ifPresent(premissa -> {
                    throw new IllegalArgumentException("Premissa não atendida: " + premissa.descricao());
                });
    }
}
```

4. Resultado da suíte NIST

A sequência de 1.000.000 de bits foi testada no testador indicado no enunciado (random-bitstream-tester), que implementa 14 testes da NIST SP 800-22. O critério de aprovação é p-value ≥ 0,01 (α = 1%). Resultado: **13 de 14 testes aprovados**; o único “Failed” é o Runs Test, explicado na seção 5.

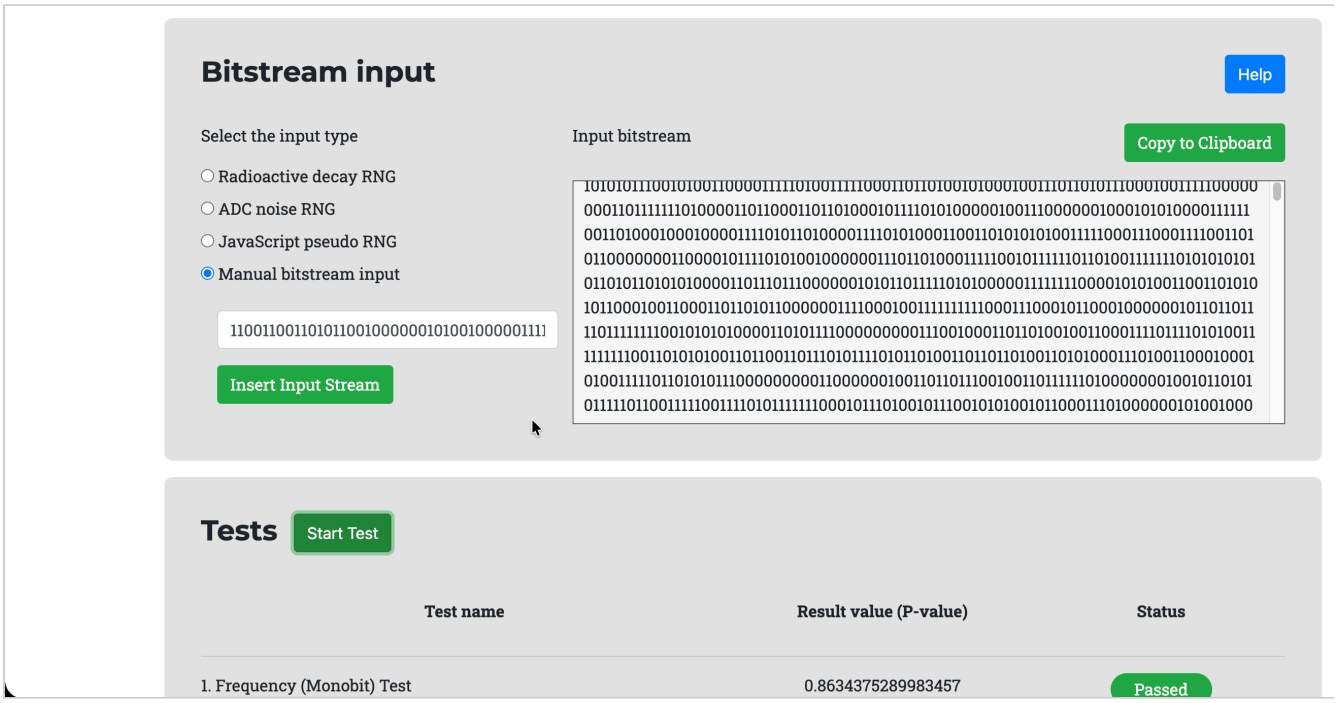


Figura 3 — Os bits gerados inseridos no testador (“Manual bitstream input”).

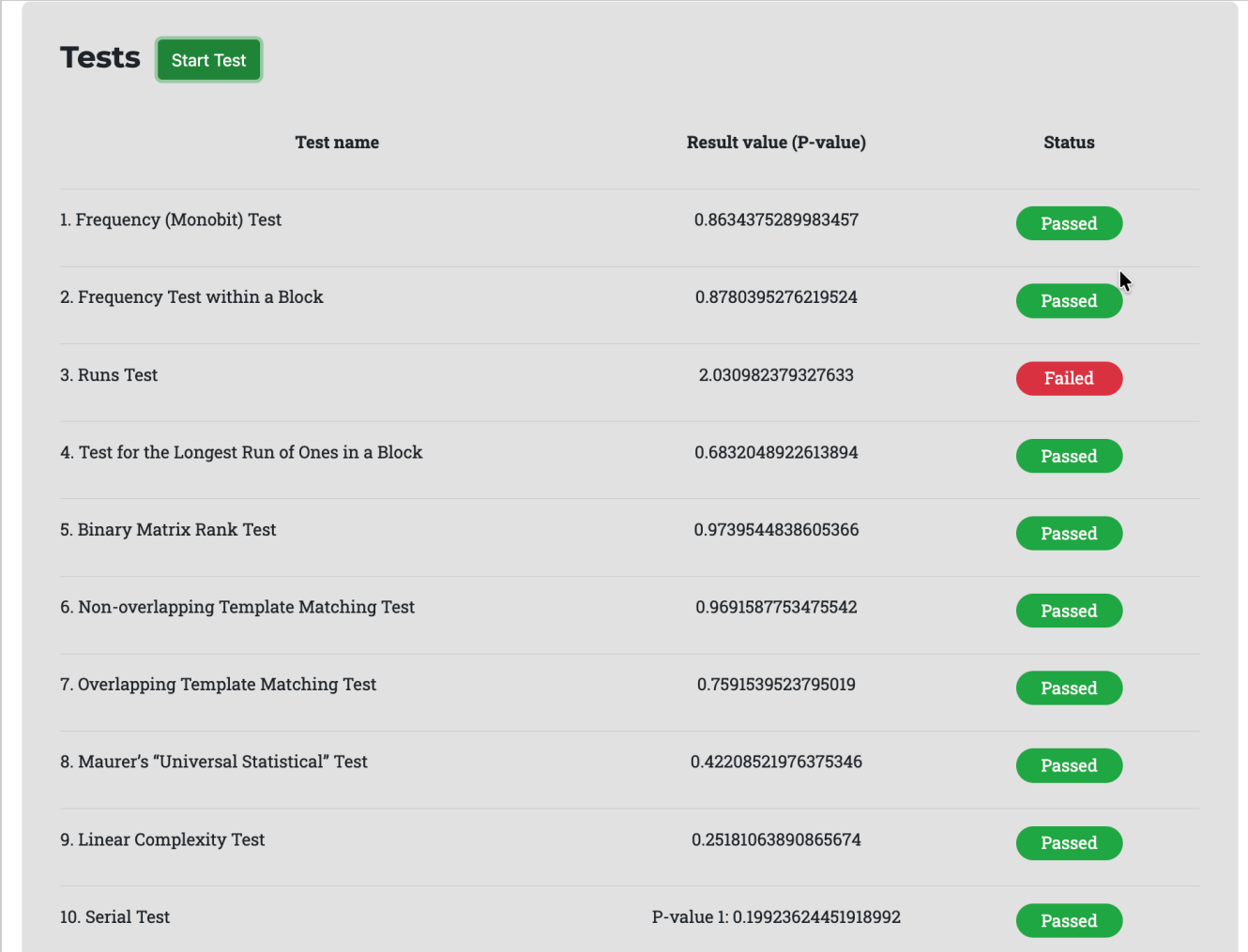


Figura 4 — Resultados dos testes 1 a 10.

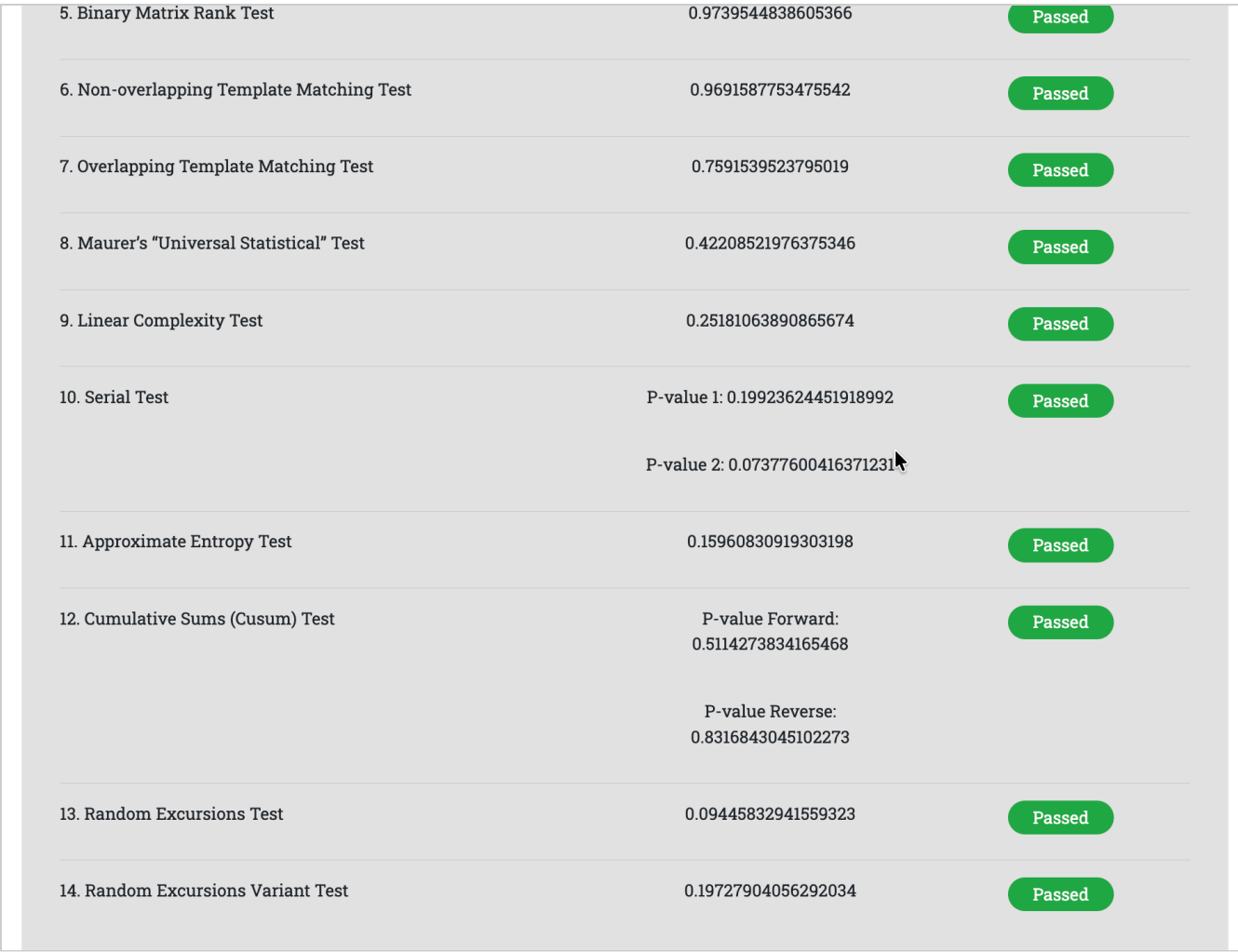


Figura 5 — Resultados dos testes 5 a 14.

5. O "Failed" no Runs Test é um erro do testador

Um p-value é uma probabilidade e, portanto, está sempre entre 0 e 1. O valor **2,031** exibido no Runs Test é matematicamente impossível: o "Failed" decorre de um erro de cálculo do testador online, que declara não ser a implementação oficial do NIST. Pela fórmula da SP 800-22 (seção 2.3), para estes bits:

```
π = 500.086 / 1.000.000 = 0,500086      pré-teste: |π − 1/2| = 0,000086 < 2/√n = 0,002  ✓
V_obs (total de sequências) = 499.106    esperado: 2nπ(1 − π) ≈ 500.000
p-value = erfc( |V_obs − 2nπ(1 − π)| / (2·√(2n)·π(1 − π)) ) = erfc(1,2643) = 0,0738 ≥ 0,01  → aprovado
```

Para confirmar, o mesmo arquivo foi testado na implementação oficial do NIST em C (sts-2.1.2), com parâmetros padrão. O Runs Test resultou em **p-value = 0,073781 (SUCCESS)**, e todos os testes foram aprovados:

Teste (NIST STS 2.1.2)	**p-value**	**Teste (NIST STS 2.1.2)**	**p-value**
Frequency	0,8634	Universal	0,4221
BlockFrequency	0,8780	LinearComplexity	0,8310
Runs	**0,0738**	Serial	0,9770 / 0,8616
LongestRun	0,6832	ApproximateEntropy	0,9082
Rank	0,9740	CumulativeSums	0,5114 / 0,6634
FFT (Spectral)	0,2329	RandomExcursions (8 estados)	todos ≥ 0,0945
OverlappingTemplate	0,7592	RandomExcursionsVariant (18)	todos ≥ 0,1973
NonOverlappingTemplate: 144 de 148 templates aprovados		(≈ 1,5 reprovações são esperadas ao acaso com α = 1%)	

Nos testes em que o testador online e o STS oficial usam a mesma configuração (Frequency, BlockFrequency, LongestRun, Rank, Overlapping, Universal), os p-values coincidem. Nos demais, diferem porque o testador online usa tamanhos de bloco que não informa.

6. Conclusão

A implementação atende às premissas do algoritmo e a sequência gerada passou nos testes estatísticos da NIST SP 800-22. Isso, porém, não torna este gerador seguro. A segurança do BBS depende da dificuldade de fatorar n, e um n de 34 bits é fatorado em milissegundos — a partir de p e q, a sequência inteira pode ser recalculada; na prática, o BBS exige n de 2048 bits ou mais. Além disso, testes estatísticos medem viés, não previsibilidade: geradores totalmente previsíveis também podem passar em todos eles. Passar na suíte NIST é condição necessária, mas não suficiente, para um gerador criptográfico. Por fim, a semente publicada aqui por exigência do enunciado seria, em uso real, secreta.

Referências

[1] L. Blum, M. Blum, M. Shub. *A Simple Unpredictable Pseudo-Random Number Generator*. SIAM Journal on Computing, 15(2), 1986.

[2] A. Rukhin et al. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*. NIST SP 800-22 Rev. 1a, 2010.

[3] Z. Molnár. *Random Bitstream Tester* (implementação JavaScript dos testes da SP 800-22). https://mzsoltmolnar.github.io/random-bitstream-tester/

[4] NIST. *Statistical Test Suite*, versão 2.1.2 (implementação de referência em C).